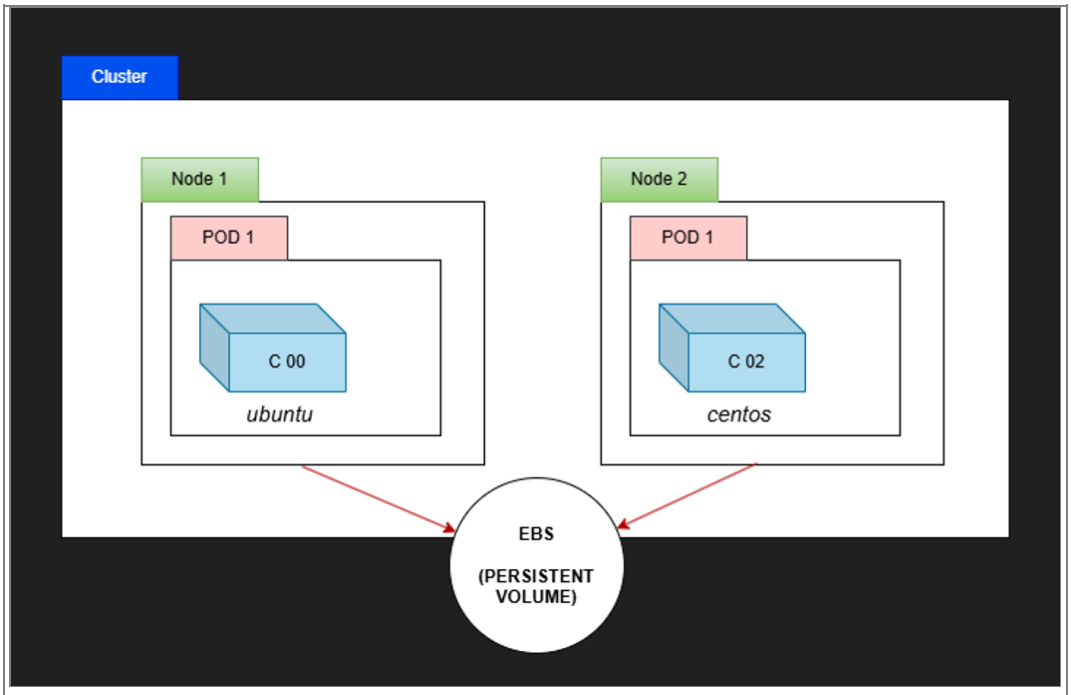
PersistentVolume in Kubernetes

Rohit Surwade

PersistentVolume (PV) in Kubernetes

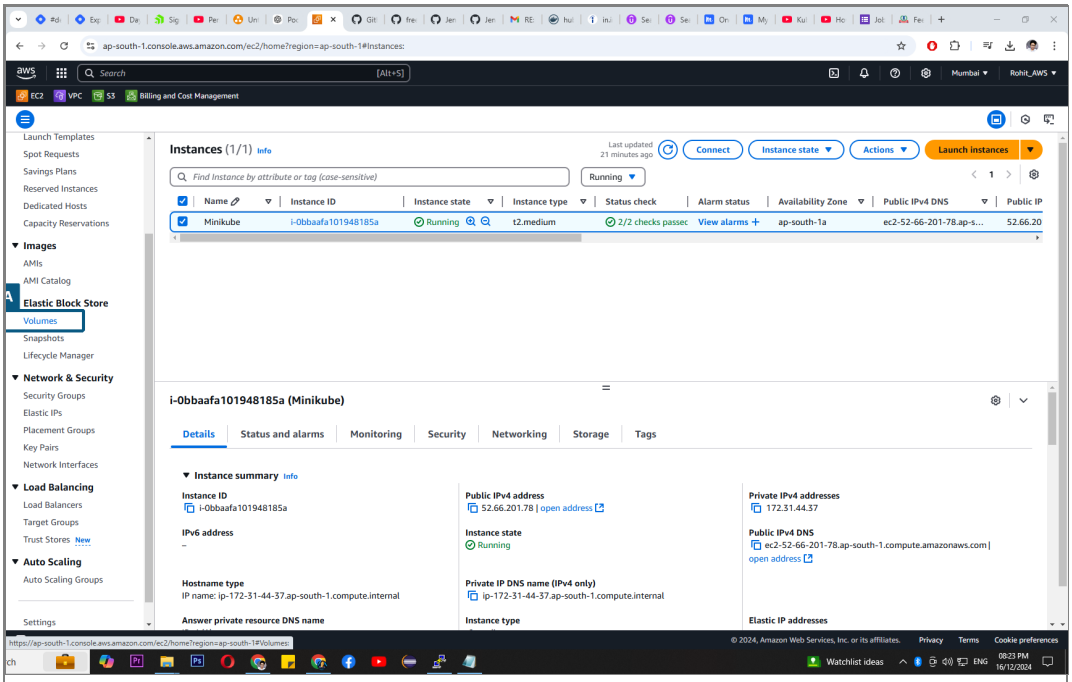


1.

PersistentVolumes in Kubernetes allow Pods to have access to durable storage that outlives individual Pods and can be shared or retained across Pod restarts. They are a crucial part of managing stateful workloads in Kubernetes, providing reliable and persistent storage for applications that need to store data beyond the Pod lifecycle.

A **PersistentVolume (PV)** in Kubernetes is a piece of storage that has been provisioned and made available for use by a Kubernetes cluster. It abstracts the underlying storage hardware or storage service (e.g., local storage, NFS, cloud storage like AWS EBS or Google Persistent Disk) and presents it in a standardized way to Kubernetes Pods. This abstraction allows Kubernetes workloads to access persistent storage without needing to worry about the specifics of how and where the storage is provisioned.

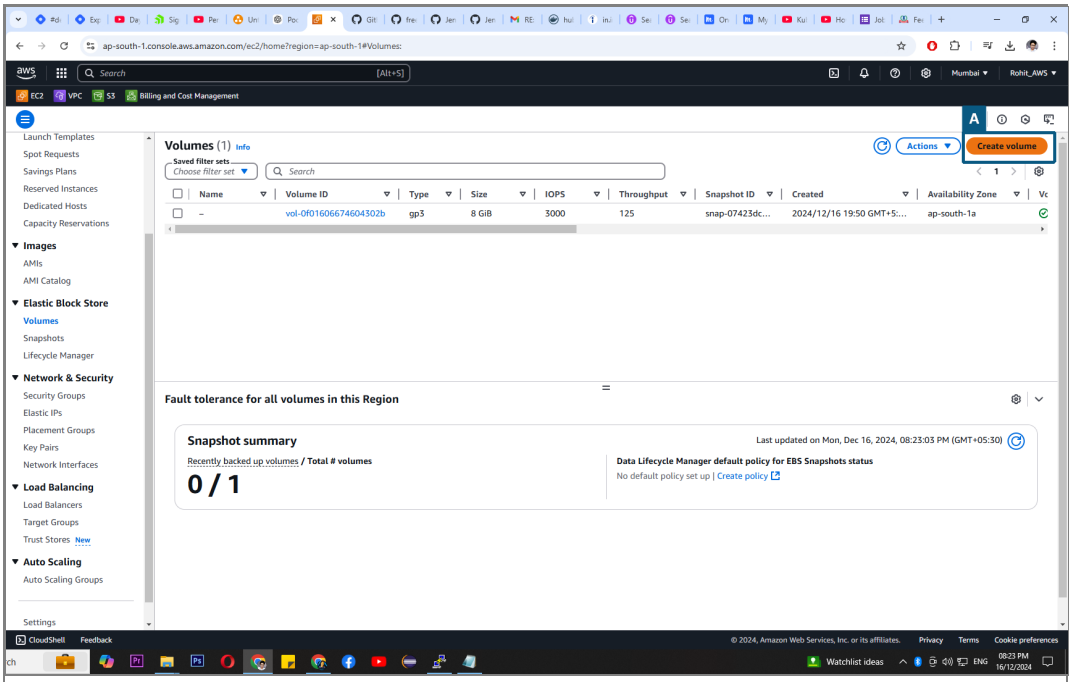
Creating an AWS EBS Volume



2.

Click on 'Volumes' under “Elastic Block Store”

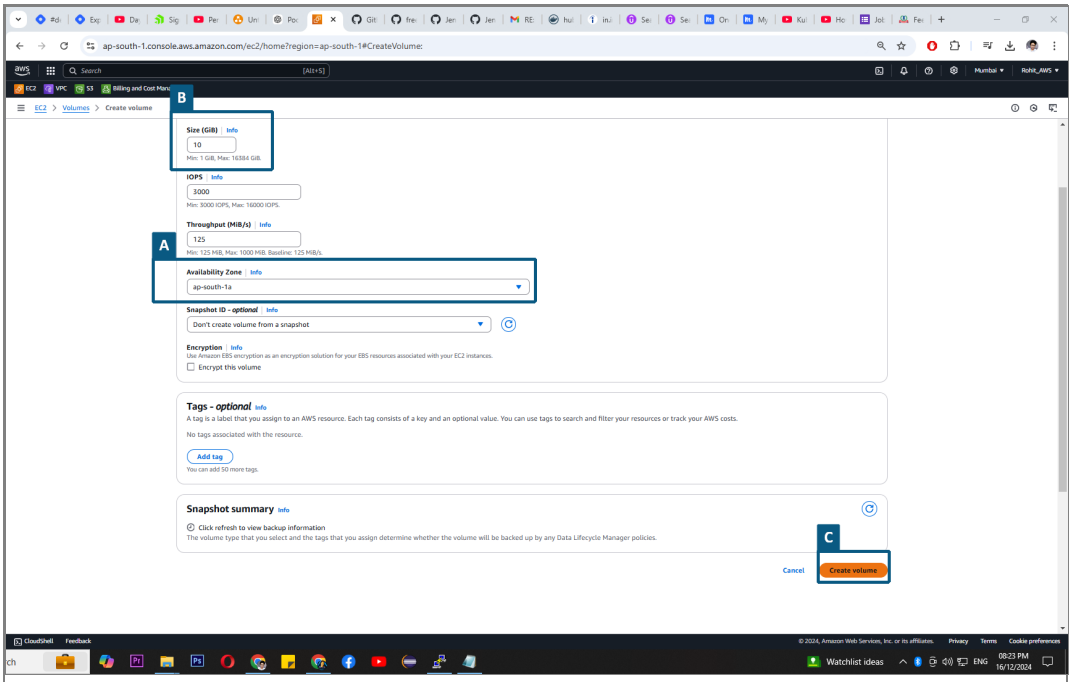
Creating an AWS EBS Volume



3.

Click on “Create Volume”

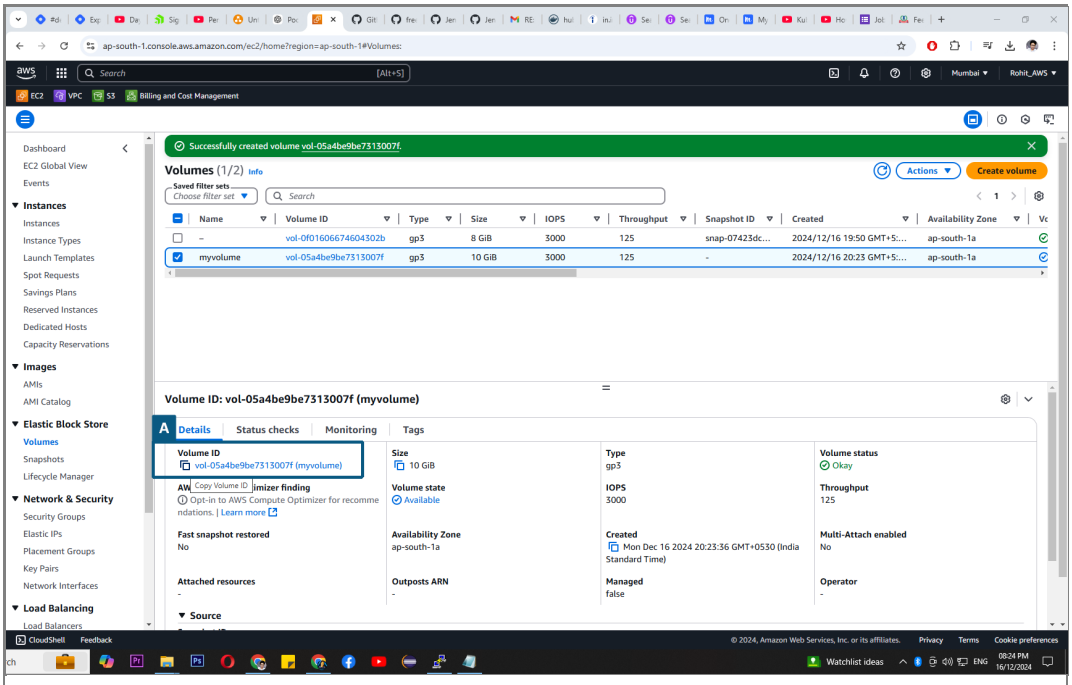
Creating an AWS EBS Volume



4.

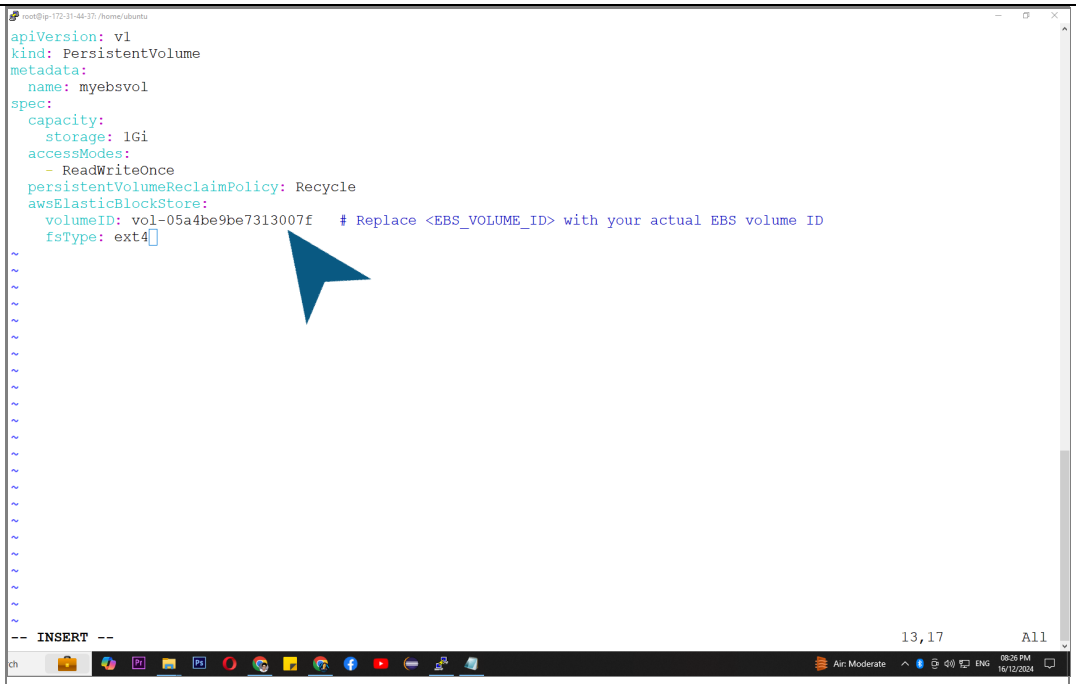
NOTE: The EBS volume that we are creating should be in the Same Region and Same Availibility Zone as the EC2 instance(i.e Node).

Copy the Volume ID of the AWS EBS volume we just created. We are gonna need it while creating the PV manifest.



5.

Manifest to create PERSISTENT VOLUME(PV)

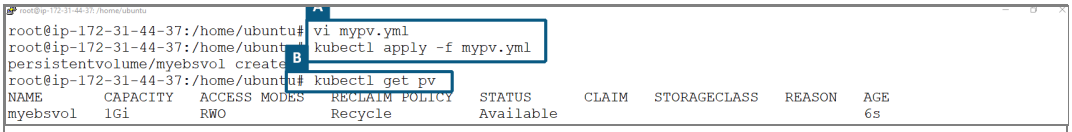


6.

(This manifest defines a PersistentVolume (PV) in Kubernetes, backed by an Amazon EBS volume, with the following details:
1. Name:
- The PV is named `myebsvol`.
2. Capacity:
- Allocates 1Gi (1 Gibibyte) of storage.
3. Access Modes:
- `ReadWriteOnce`: The volume can be mounted as read/write by a single node at a time.
4. Reclaim Policy:
- `Recycle`: When the volume is released, Kubernetes erases its data and makes it available for reuse.
5. Backend Configuration:
- Uses an AWS EBS volume identified by `<EBS_VOLUME_ID>`.
- Configured with an `ext4` file system for storing and organizing data.
Purpose:
This PV provides durable, reusable storage that can be claimed by a PersistentVolumeClaim (PVC) and used by a Pod.
It is particularly suited for workloads running on AWS.)

apiVersion: v1
kind: PersistentVolume
metadata:
name: myebsvol
spec:
capacity:
storage: 1Gi
accessModes:
- ReadWriteOnce
persistentVolumeReclaimPolicy: Recycle
awsElasticBlockStore:
volumeID: vol-05a4be9be7313007f # Replace <EBS_VOLUME_ID> with your actual EBS volume ID
fsType: ext4

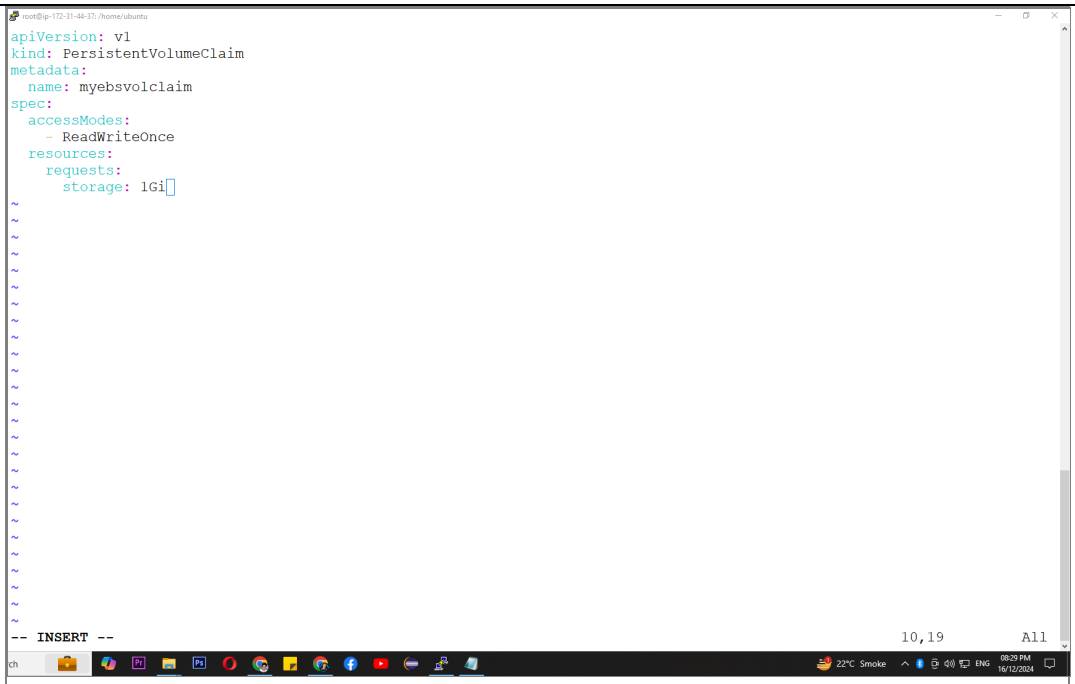
List the successfully Created PV.



7.

commands:
kubectl get pv

Manifest to create PersistentVolumeClaim (PVC).

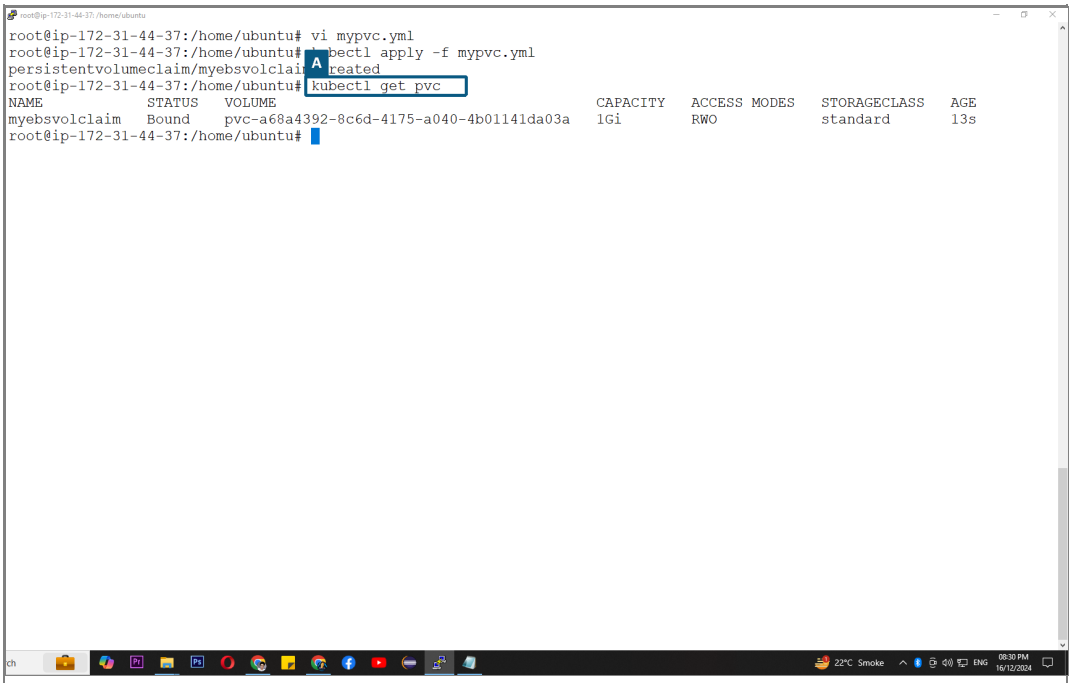


8.

(This manifest defines a PersistentVolumeClaim (PVC) named `myebsvolclaim` that requests 1Gi of storage with the following specifications:
1. Access Mode:
- `ReadWriteOnce`: The volume will be mounted as read/write by a single node.
2. Storage Request:
- Asks for 1Gi of storage from a compatible PersistentVolume (PV).
Purpose:
The PVC enables a Pod to dynamically or statically claim storage from a PV that matches the requested size and access mode. Once bound, the Pod can use the claimed storage for persistent data.)

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
name: myebsvolclaim
spec:
accessModes:
- ReadWriteOnce
resources:
requests:
storage: 1Gi

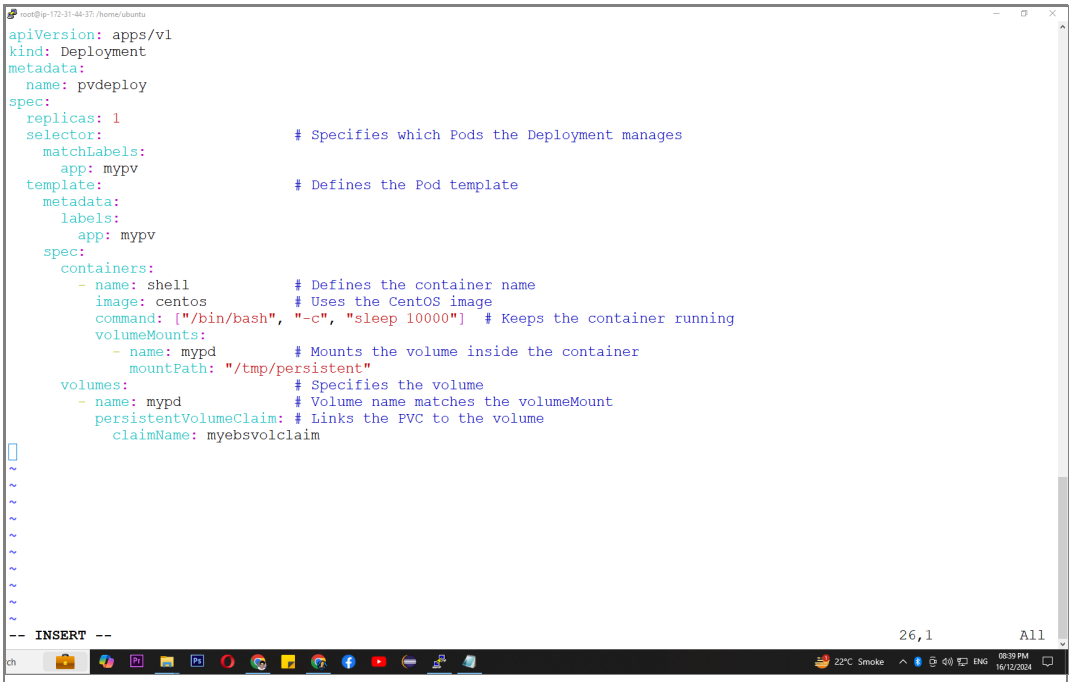
List the successfully Created PersistentVolumeClaim(PVC).



9.

commands:
kubectl get pvc

Deployment Manifest to test functioning of PV and PVC.

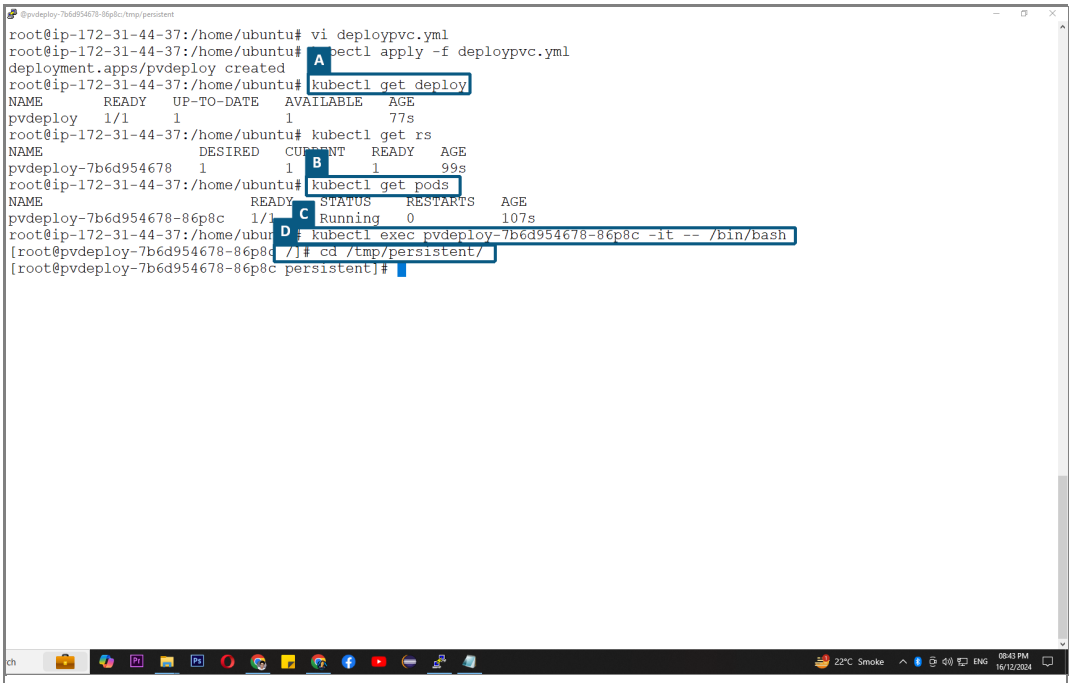


10.

(This Kubernetes Deployment manifest defines a Deployment resource named pvdeploy, which manages a single replica (Pod) of a CentOS container.
The container runs a long-running bash command (sleep 10000), which ensures that the container stays alive for a specified duration,
simulating an active task. The Deployment ensures the Pod remains in the desired state and will automatically replace it if it fails or gets terminated.
Additionally, the container mounts a volume named mypd to the /tmp/persistent directory inside the container.
This volume is backed by a PersistentVolumeClaim (PVC) called myebsvolclaim, which ensures that the container has access to durable storage.
The PVC links to an existing PersistentVolume (PV), which could be provisioned on various storage backends like AWS EBS.
The volume provides persistent storage, meaning any data written to /tmp/persistent will be retained even if the container or Pod is deleted or restarted.
The Deployment ensures that the Pod, along with its persistent storage, is consistently available and managed, making this setup ideal for applications that require long-term data storage beyond the Pod's lifecycle.)

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: pvdeploy
spec:
replicas: 1
selector: # Specifies which Pods the Deployment manages
matchLabels:
app: mypv
template: # Defines the Pod template
metadata:
labels:
app: mypv
spec:
containers:
- name: shell # Defines the container name
image: centos # Uses the CentOS image
command: ["/bin/bash", "-c", "sleep 10000"] # Keeps the container running
volumeMounts:
- name: mypd # Mounts the volume inside the container
mountPath: "/tmp/persistent"
volumes: # Specifies the volume
- name: mypd # Volume name matches the volumeMount
persistentVolumeClaim: # Links the PVC to the volume
claimName: myebsvolclaim
```

Enter the POD and head on to the volume mounted directory as per the manifest(/tmp/persistent)

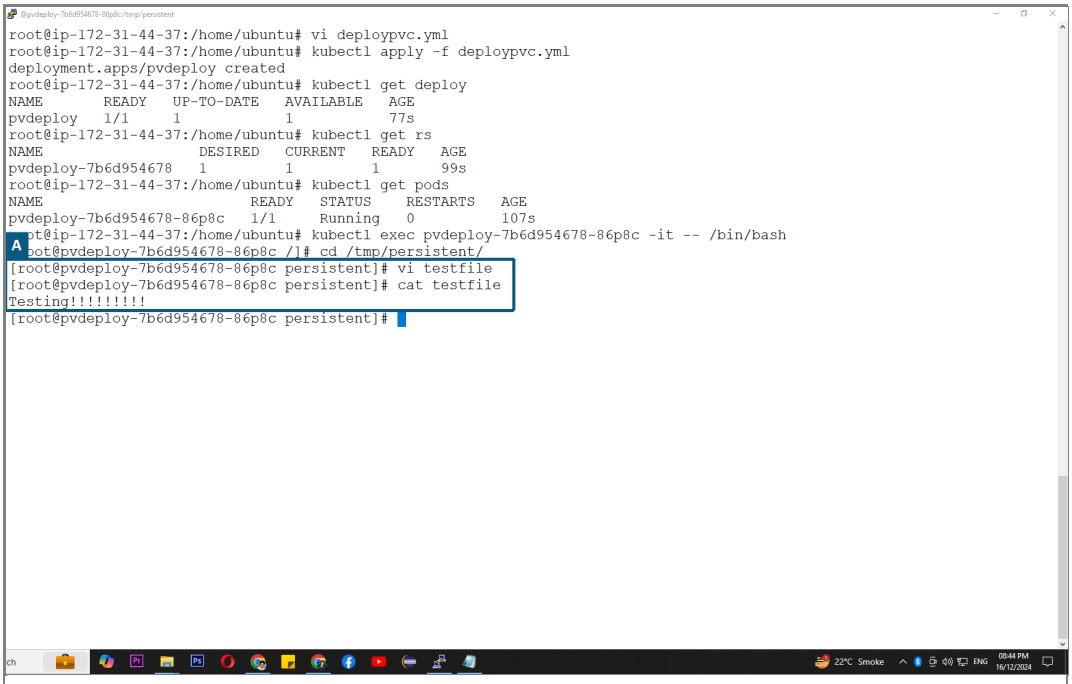


11.

commands:

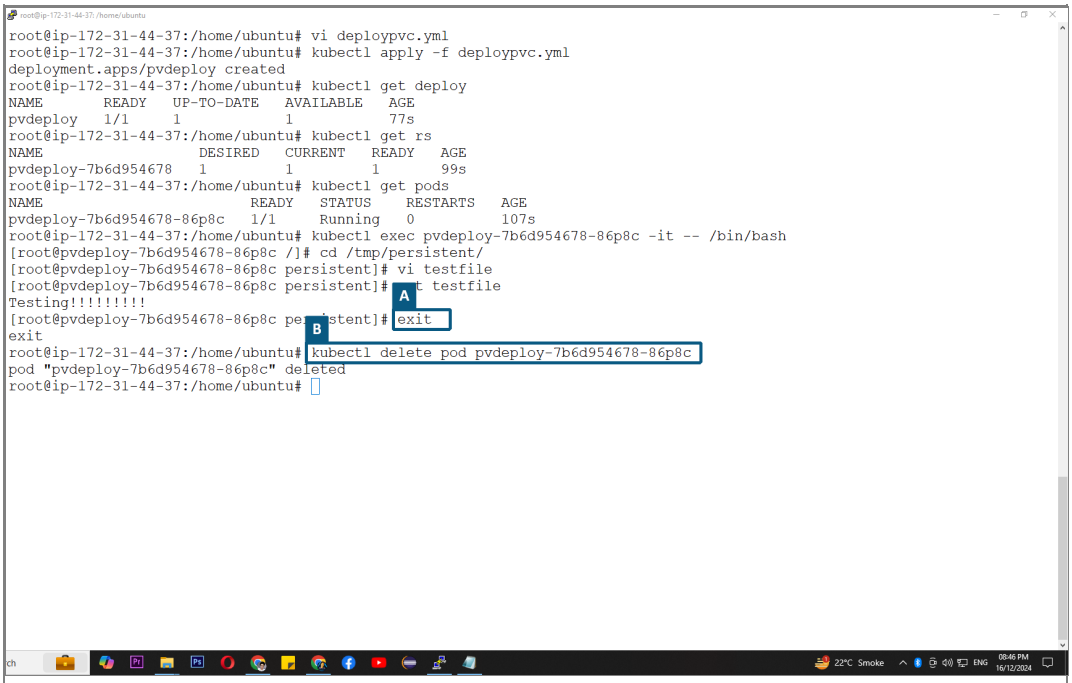
1. kubectl get pods
2. kubectl exec pod-name -it -- /bin/bash

Create a file in the PV mounted directory for testing



12.

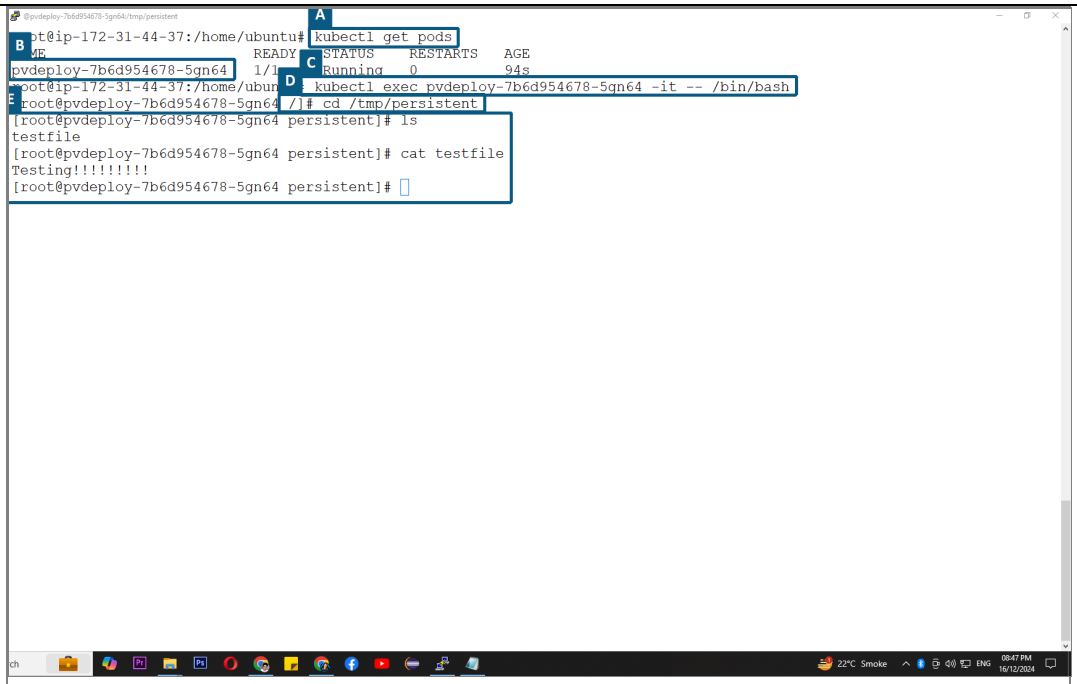
Exit the POD/Container and Delete the POD



13.

commands:
kubectl delete pod pod-name

Check if the data is reflecting in newly created POD.



14.

Data is reflecting!!

(NOTE: Deployment will maintain the desired state and create a new POD right away since all the nodes in the cluster are sharing the same EBS volume the data will reflect even in the newly created POD no matter which node it gets created in(in the cluster).)